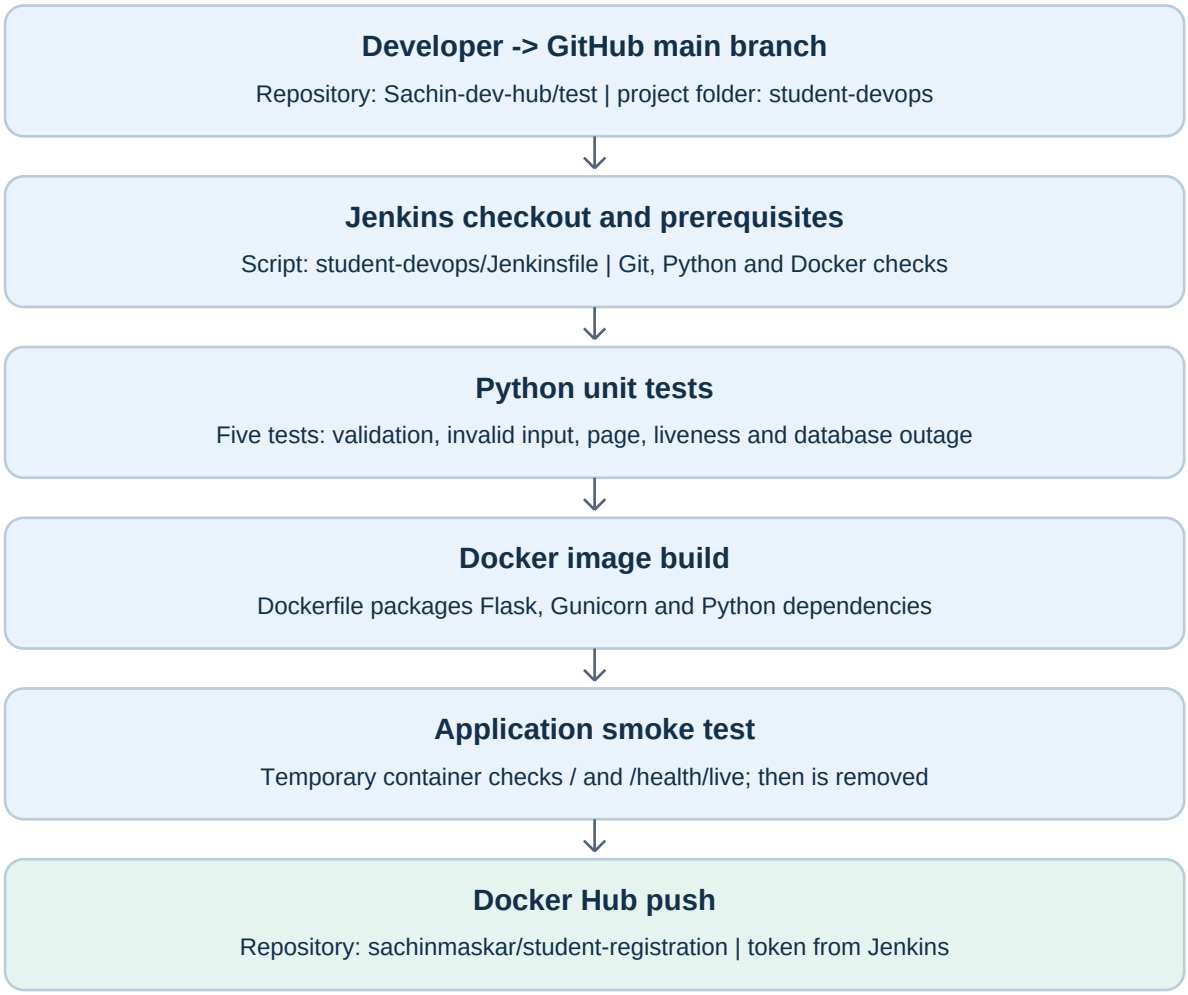
CI/CD Pipeline Architecture

Student Registration Application | GitHub + Jenkins + Docker + Docker Hub

Current working pipeline

Jenkins runs on your Ubuntu VM in Oracle VirtualBox. It reads the Jenkinsfile from GitHub, runs tests, builds an image, checks application startup and publishes the image to Docker Hub.



Build trigger

Build Now starts the job manually. Optional Poll SCM (H/5 * * * *) checks for new commits approximately every five minutes. A webhook is not required for polling.

Scope: build and image publishing are working. Automatic deployment to your escbash Kubernetes cluster is not yet part of this pipeline.

Inside the Jenkins VM

Stage	What happens	Success condition
Checkout	Jenkins checks out main and reads the Git commit ID.	Expected source revision is checked out.
Prerequisites	Verifies Git, Python, Docker daemon access, Dockerfile and tests.	Commands and file checks pass.
Python tests	Creates a virtual environment and installs requirements. Runs unittest.	All five tests pass.
Build image	Runs docker build from the student-devops directory. Adds a source revision label.	A local versioned image is created.
Smoke test	Starts the image without publishing a host port. Checks the page and liveness endpoint.	Both endpoints return HTTP 200.
Push	Logs in with Jenkins credentials and pushes the generated tag.	Registry returns an image digest.

Image identification

```
sachinmaskar/student-registration:build-6-a1b2c3d4e5f6
```

This is an example tag: build-6 identifies the Jenkins build and the final part identifies the Git commit. The pipeline uses a distinct tag rather than replacing v2 or latest. Avoid overwriting released tags to preserve traceability.

Credentials and runtime access

Jenkins credential ID: dockerhub. Type: Username with password. Username: sachinmaskar. Password: Docker Hub token with Read & Write permission. Tokens belong in Jenkins credentials, not source files.

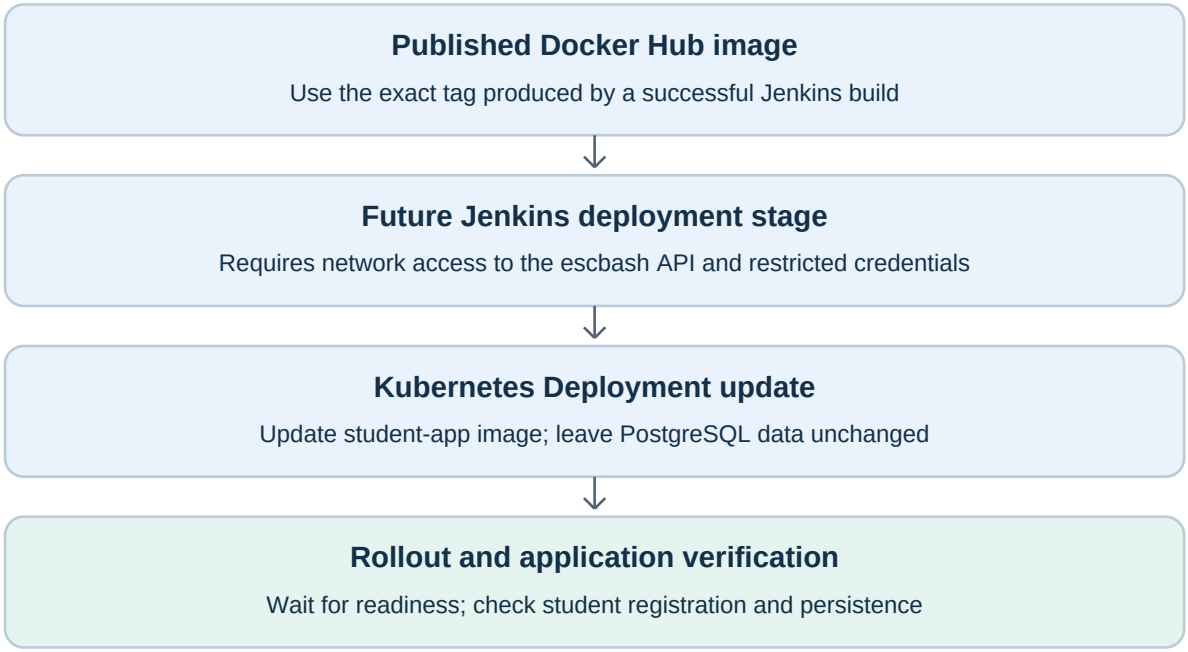
Docker login uses --password-stdin and a temporary DOCKER_CONFIG directory. The pipeline removes that directory when the push step exits. Docker group membership lets the jenkins user access Docker and grants root-level power on the VM; use trusted code in this dedicated lab.

What is not inside the image

Live PostgreSQL records and the local .env file are not packaged by the supplied Dockerfile. Database configuration is supplied when running the container. PostgreSQL runs separately.

From published image to Kubernetes

Your current pipeline ends at Docker Hub. The following diagram shows the next deployment extension; it is a design to implement, not a confirmed automated step.



Manual deployment before adding automation

On escbash, once the database and student-app Deployment exist, replace YOUR_NEW_TAG with the actual pushed tag:

```
kubectl -n student-devops set image deployment/student-app \
app=sachinmaskar/student-registration:YOUR_NEW_TAG
kubectl -n student-devops rollout status \
deployment/student-app --timeout=180s
```

Public images can be pulled directly; private images need an imagePullSecret. Your Jenkins VM and escbash are separate machines. Do not assume the VM can reach a localhost-only kind API endpoint on escbash.

Failure handling

A failed test or build skips later stages. A failed push means there is no confirmed published release. Login Succeeded followed by insufficient scopes means the token lacks repository push permission. A Dockerfile COPY error mentioning # is fixed by putting comments on separate lines.

No automatic rollback is configured. After a failed Kubernetes release, inspect rollout history and use rollout undo if an earlier revision and image are available. This does not restore database records or schema.

Next improvements

Add real PostgreSQL integration tests, image vulnerability scanning, restricted deployment credentials and a reachable deployment route. Then add readiness checks and functional validation to complete automated delivery. The current smoke test does not validate database CRUD.